Colorizing the Prokudin-Gorskii Collection

CS180 Project 1 · John Chan

1. Introduction

Between 1909 and 1915, Sergei Mikhailovich Prokudin-Gorskii travelled across the Russian Empire with a special camera. For every scene he exposed three black-and-white glass plates in quick succession, each through a different color filter: **blue**, **green** and **red**. He never saw his photographs in color the way we can today, but the plates survived and were digitized by the Library of Congress.

Each digitized plate stacks the three exposures vertically (B on top, then G, then R). Because the camera and the subject moved slightly between exposures, simply cutting the plate into thirds and stacking them as RGB gives a blurry image with strong color fringes. The goal of this project is to **find the (dx, dy) shift that aligns G and R to B** automatically, and produce a clean color image.



The raw plate: B, G, R from top to bottom.



Baseline: cut into thirds and stacked with no alignment → color fringing everywhere.

2. Single-scale alignment

2.1 Splitting the plate

The image is read and converted to float in $[0, 1]$ right away (TIFs are 16-bit, JPGs 8-bit, so this puts both on the same scale). The height is divided by 3 and the plate is cut into three equally sized channels. Each channel has a different overall brightness, because each filter lets through a different amount of light. This matters for choosing a metric.



The three channels of `cathedral.jpg`. Same scene, different brightness (mean 0.45 / 0.54 / 0.58).

2.2 Scoring an alignment: L2 vs. NCC

To compare a shifted channel A with the reference channel B we need a score.

- **L2 / Sum of Squared Differences** — lower is better:

$$\mathrm{SSD}(A, B) = \sum_{x,y} \big(A(x,y) - B(x,y)\big)^2$$

  Simple and fast, but it assumes the two channels have the *same brightness*. If one channel is globally brighter, every pixel adds to the error even when the alignment is perfect.
- **Normalized Cross-Correlation (NCC)** — higher is better (max 1):

$$\mathrm{NCC}(A, B) = \frac{\sum (A - \bar{A})(B - \bar{B})}{\|A - \bar{A}\| \, \|B - \bar{B}\|}$$

Subtracting the mean removes brightness offsets, and dividing by the norm removes contrast differences. So NCC is unchanged if $B = \alpha A + \beta$ with $\alpha > 0$, which is exactly the kind of difference we see between the filtered channels.

Both metrics are implemented; NCC is the default for all results below.

2.3 Ignoring the borders

The plates have black and white borders, scratches and uneven edges that are *different* in each channel. Also, I shift images with `np.roll`, which wraps pixels around from one side to the other. If the score included the edges, these artifacts would dominate it. So the score is computed only on the **interior: 10% is cropped from every side** before comparing. Using a fraction (not a fixed number of pixels) means the same setting works for 400 px JPGs and 3000 px TIFs.

2.4 Exhaustive search

For each candidate shift $(dx, dy) \in [-15, 15]^2$ (961 candidates), shift the moving channel, crop both, and compute the score. Keep the shift with the best score. B is the reference; G and R are each aligned to B independently.

```
for dy in −15..15:
    for dx in −15..15:
        s = NCC( crop(shift(moving, dx, dy)), crop(ref) )
        keep (dx, dy) with the highest s
```

2.5 Results on the small JPGs

All three JPGs align cleanly. **L2 and NCC give exactly the same offsets** on these images, which makes sense: their channels differ mostly by brightness, and after cropping the borders, both metrics find the same peak. Each search takes about 0.4–0.7 s.



cathedral · G (2, 5) · R (3, 12)



monastery · G (2, –3) · R (2, 3)



tobolsk · G (3, 3) · R (3, 6)

image	metric	G (dx, dy)	R (dx, dy)	time (s)
cathedral.jpg	L2	(2, 5)	(3, 12)	0.42
cathedral.jpg	NCC	(2, 5)	(3, 12)	0.67
monastery.jpg	L2	(2, –3)	(2, 3)	0.35
monastery.jpg	NCC	(2, –3)	(2, 3)	0.63
tobolsk.jpg	L2	(3, 3)	(3, 6)	0.36
tobolsk.jpg	NCC	(3, 3)	(3, 6)	0.58

Offsets are in pixels, (dx, dy) = (right, down), relative to the B channel.

3. Multi-scale pyramid

3.1 Why single-scale is not enough

The TIF scans are about **3200 × 3600 px per channel**, roughly 70× more pixels than the JPGs. The offsets are also much bigger: up to ~180 px for R. To find that with brute force we would need a $[-180, 180]^2$ window, i.e. about 130,000 candidates, each scored on millions of pixels. Extrapolating from my JPG timing, that would take **hours per image**.

3.2 Building the pyramid (with anti-aliasing)

An image pyramid is a stack of the same image at decreasing resolutions, each level half the size of the one below. To go down one level I average every 2 × 2 block of pixels (a box filter) instead of just taking every second pixel. Taking every second pixel throws away information and lets fine detail (grass, fabric, film grain) *alias* into fake coarse patterns, which can mislead the search at the coarse levels. Averaging first acts as a low-pass filter and removes those frequencies.



Every 2nd pixel, 3 times (÷8): **noisy, grainy**



2 × 2 average, 3 times (÷8): **smooth, faithful**

Levels are added until the next one would have a short side under 150 px. For a TIF this gives 5 levels:

level	scale	size (church, B)	search window	covers at full res
4 (coarsest)	1/16	200 × 227	±15	±240 px
3	1/8	400 × 454	±3 around 2 × previous	refine
2	1/4	800 × 908	±3 around 2 × previous	refine
1	1/2	1601 × 1817	±3 around 2 × previous	refine
0 (original)	1	3202 × 3634	±3 around 2 × previous	final offset

3.3 Coarse-to-fine search

1. Build pyramids for both the moving channel and the reference.
2. At the **coarsest** level, run the exhaustive search with a ±15 window. At 1/16 scale, that covers ±240 px in the original, more than any offset in the dataset.
3. Go up one level: **double** the offset (every pixel is now two pixels), then search only a small ±3 window around it to fix the rounding error from the coarser level.
4. Repeat until the original resolution. The last refined offset is the answer.

```
dx, dy = 0, 0
for level from coarsest to finest:
    window = 15 if coarsest else 3
    dx, dy = best shift in window around (dx, dy) at this level
    if not finest: dx, dy = 2*dx, 2*dy
```

3.4 Parameters and running time

One fixed set of parameters is used for **every** image, with no per-image tuning:

- **Interior crop:** 10% per side (same as single-scale).
- **Coarsest level:** short side ≥ 150 px. Small enough that a ±15 search is cheap, big enough that the image still has real structure to match.
- **Coarse window ±15:** covers ±240 px at full resolution.
- **Fine window ±3:** an error of ±1 px at the coarser level becomes ±2 px after doubling, so ±3 is enough with a small margin. Keeping it small matters most at the finest level, where each score costs the most.

Result: each TIF (both G and R) is aligned in **about 5–6 seconds** on my laptop, instead of hours. On the JPGs the pyramid gives exactly the same offsets as the single-scale search, which is a good sanity check.

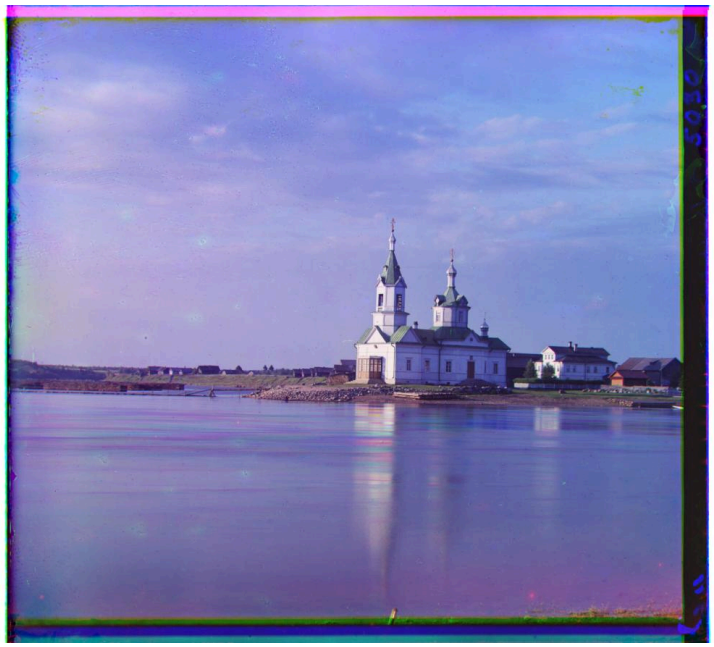
4. Results on all images

All results use the **basic method: NCC on raw RGB pixel values**, B as reference, pyramid with the parameters above. **13 / 14** images align correctly; `emir` fails in R (see section 5).

image	method	G (dx, dy)	R (dx, dy)	time (s)
cathedral.jpg	pyramid	(2, 5)	(3, 12)	0.20
monastery.jpg	pyramid	(2, −3)	(2, 3)	0.24
tobolsk.jpg	pyramid	(3, 3)	(3, 6)	0.33
church.tif	pyramid	(4, 25)	(−4, 58)	5.27
emir.tif	pyramid	(24, 49)	(−205, 141) ✕	5.71

harvesters.tif	pyramid	(17, 60)	(13, 124)	5.79
icon.tif	pyramid	(17, 41)	(23, 89)	5.94
ilemselga.tif	pyramid	(7, 40)	(11, 130)	6.09
melons.tif	pyramid	(11, 82)	(13, 178)	6.06
religous_painting.tif	pyramid	(3, 28)	(7, 68)	5.69
self_portrait.tif	pyramid	(29, 79)	(37, 176)	5.94
siren.tif	pyramid	(−6, 49)	(−25, 96)	5.79
three_generations.tif	pyramid	(14, 53)	(11, 112)	5.74
wharf.tif	pyramid	(−7, 15)	(−16, 83)	5.78

Note how the R offset is usually about twice the G offset: the three plates were exposed one after another, so whatever drifted (camera or plate holder) kept drifting in the same direction.



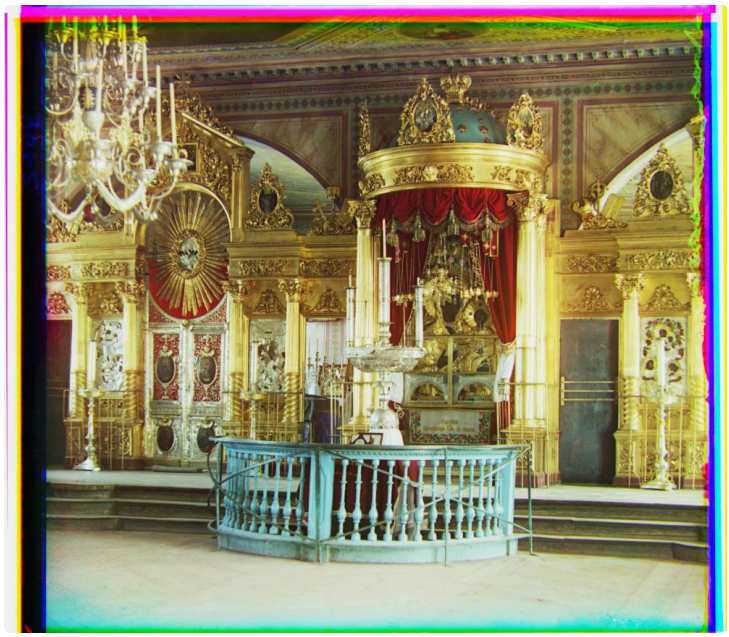
church · G (4, 25) · R (−4, 58)



emir · G (24, 49) · R (−205, 141) · **R fails**



harvesters · G (17, 60) · R (13, 124)



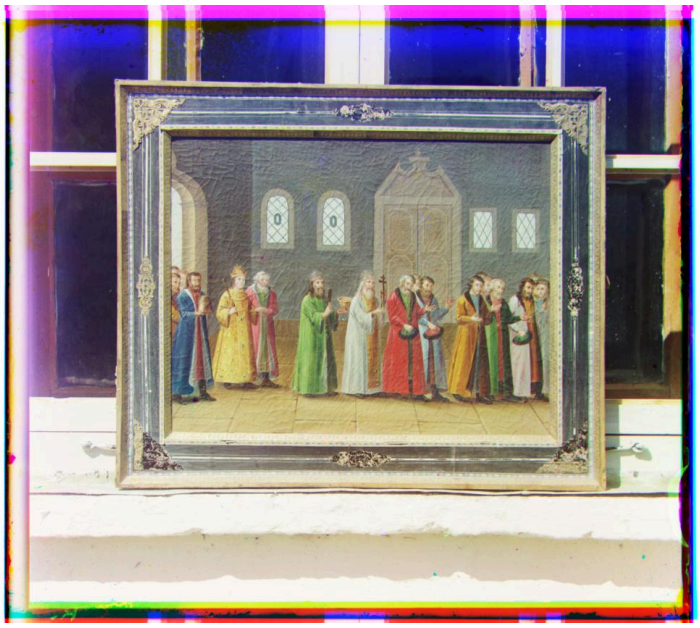
icon · G (17, 41) · R (23, 89)



ilemselga · G (7, 40) · R (11, 130)



melons · G (11, 82) · R (13, 178)



religous_painting · G (3, 28) · R (7, 68)



self_portrait · G (29, 79) · R (37, 176)



siren · G (−6, 49) · R (−25, 96)



three_generations · G (14, 53) · R (11, 112)



wharf · G (–7, 15) · R (–16, 83)

4.1 Additional images from the Library of Congress

I also downloaded plates from the Prokudin-Gorskii Collection and ran them through the **same code with the same parameters** (no tuning). They align cleanly; single-scale and pyramid give identical offsets.

image (LoC id)	method	G (dx, dy)	R (dx, dy)	time (s)
prok-00000-00016v	pyramid	(1, 4)	(2, 10)	0.27
prok-00200-00276v	pyramid	(1, 4)	(2, 14)	0.26



prok-00000-00016v · G (1, 4) · R (2, 10)



prok-00200-00276v · G (1, 4) · R (2, 14)

5. Failure analysis: emir

With RGB NCC, G aligns fine, but R lands at **(–205, 141)**, far away from where it should be. The zoom below shows a whole second, shifted copy of the Emir in cyan and red.



RGB NCC, zoomed in: the R channel is placed ~200 px off, producing a ghost.

Why it fails. NCC only tolerates a *linear* brightness relationship between channels, $R \approx \alpha B + \beta$. The Emir's robe is bright blue: it is **very bright in B and very dark in R**, while other areas (the wall, his face) are bright in R. So in some regions B and R are positively related and in others negatively related. No single α, β explains both. The best "match" in raw pixel values is then not the true alignment, and because the error happens at the coarsest pyramid level, the ±3 refinement steps cannot recover from it.

G sits between B and R in wavelength, so G–B looks more alike and is still aligned correctly.

6. Bells & Whistles: gradient features

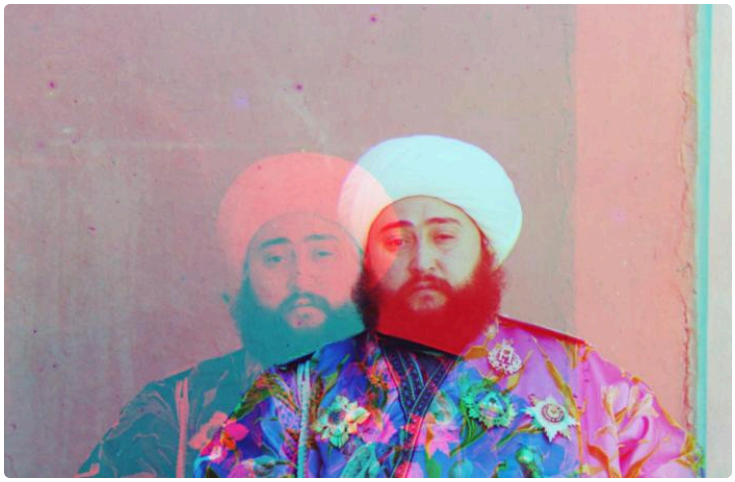
Motivation. Brightness differs between channels, but *edges* are in the same place: the outline of the robe is where it is no matter which filter you look through. So instead of comparing pixel values, compare edges.

Method. Before scoring, each pyramid level is converted to its **gradient magnitude** using Sobel filters:

$$\|\nabla I\| = \sqrt{(\partial_x I)^2 + (\partial_y I)^2}$$

The rest of the pipeline is unchanged (same NCC, same crop, same pyramid and windows). Taking the magnitude also removes the sign, so "dark-to-bright" and "bright-to-dark" edges look the same. This is the `--feature grad` option in my code.

Result. emir's R offset goes from (–205, 141) to **(40, 107)**, consistent with G (24, 49) and with the "R ≈ 2 × G" pattern seen on the other plates.



Before · RGB NCC · R (–205, 141)



After · gradient NCC · R (40, 107)



Full image, RGB NCC



Full image, gradient NCC

Gradient features do not break anything else. On the other 13 images, the offsets stay within 2 px of the RGB result (visually identical), so all **14 / 14** images align:

image	G · RGB	G · gradient	R · RGB	R · gradient
church	(4, 25)	(4, 25)	(−4, 58)	(−4, 58)
emir	(24, 49)	(24, 49)	(−205, 141)	(40, 107)
harvesters	(17, 60)	(17, 60)	(13, 124)	(14, 124)
icon	(17, 41)	(17, 42)	(23, 89)	(23, 90)
ilemselga	(7, 40)	(7, 40)	(11, 130)	(11, 131)
melons	(11, 82)	(10, 80)	(13, 178)	(13, 177)
religous_painting	(3, 28)	(5, 30)	(7, 68)	(6, 70)
self_portrait	(29, 79)	(29, 78)	(37, 176)	(37, 176)
siren	(−6, 49)	(−6, 49)	(−25, 96)	(−24, 96)
three_generations	(14, 53)	(12, 54)	(11, 112)	(9, 111)
wharf	(−7, 15)	(−7, 15)	(−16, 83)	(−16, 83)

The cost is small: computing the Sobel filters adds about 1 s per TIF (≈ 7 s instead of ≈ 6 s).

7. What I learned

- **The metric encodes an assumption.** L2 assumes equal brightness, NCC assumes a linear relation, gradients assume only that edges line up. emir showed exactly where the NCC assumption breaks.
- **Borders matter more than expected.** Without cropping the interior, the plate edges and the `np.roll` wrap-around can dominate the score.
- **Coarse-to-fine is a huge speed-up**, from hours to seconds, but it has a weak point: if the coarsest level is wrong, the small refinement windows cannot fix it.
- **Downsample properly.** Averaging before subsampling keeps the coarse levels faithful to the real image.
- Analyzing a failure (and asking which assumption broke) taught me more than the images that worked.

Images: Prokudin-Gorskii Collection, Library of Congress. Offsets are (dx, dy) in pixels relative to the blue channel.